

```
In [4]: import warnings
warnings.filterwarnings("ignore", category=UserWarning)
import numpy as np
from tqdm import tqdm
import torch
import torch.nn as nn
import torchani
import matplotlib.pyplot as plt
```

Use GPU

```
In [7]: device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
print(device)
print(torch.cuda.is_available())
print(torch.cuda.device_count())
```

```
cuda
True
1
```

Set up AEV computer

AEV: Atomic Environment Vector (atomic features)

Ref: Chem. Sci., 2017, 8, 3192

```
In [8]: def init_aev_computer():
    Rcr = 5.2
    Rca = 3.5
    EtaR = torch.tensor([16], dtype=torch.float, device=device)
    ShfR = torch.tensor([
        0.900000, 1.168750, 1.437500, 1.706250,
        1.975000, 2.243750, 2.512500, 2.781250,
        3.050000, 3.318750, 3.587500, 3.856250,
        4.125000, 4.393750, 4.662500, 4.931250
    ], dtype=torch.float, device=device)

    EtaA = torch.tensor([8], dtype=torch.float, device=device)
    Zeta = torch.tensor([32], dtype=torch.float, device=device)
    ShfA = torch.tensor([0.90, 1.55, 2.20, 2.85], dtype=torch.float, device=device)
    ShfZ = torch.tensor([
```

```

    0.19634954, 0.58904862, 0.9817477, 1.37444680,
    1.76714590, 2.15984490, 2.5525440, 2.94524300
], dtype=torch.float, device=device)

num_species = 4
aev_computer = torchani.AEVComputer(
    Rcr, Rca, EtaR, ShfR, EtaA, Zeta, ShfA, ShfZ, num_species
)
return aev_computer

aev_computer = init_aev_computer()
aev_dim = aev_computer.aev_length
print(aev_dim)

```

384

Prepare dataset & split

```

In [9]: def load_ani_dataset(dspath):
        self_energies = torch.tensor([
            0.500607632585, -37.8302333826,
            -54.5680045287, -75.0362229210
        ], dtype=torch.float, device=device)
        energy_shifter = torchani.utils.EnergyShifter(None)
        species_order = ['H', 'C', 'N', 'O']

        dataset = torchani.data.load(dspath)
        dataset = dataset.subtract_self_energies(energy_shifter, species_order)
        dataset = dataset.species_to_indices(species_order)
        dataset = dataset.shuffle()
        return dataset

dataset = load_ani_dataset("./ani_gdb_s01_to_s04.h5")
# Use dataset.split method to do split
train_data, val_data, test_data = dataset.split(0.8, 0.1, 0.1)

```

Batching

```

In [10]: batch_size = 8192
# use dataset.collate(...).cache() method to do batching
train_data_loader = train_data.collate(batch_size).cache()
val_data_loader = val_data.collate(batch_size).cache()
test_data_loader = test_data.collate(batch_size).cache()

```

Torchani API

```
In [11]: class AtomicNet(nn.Module):
    def __init__(self):
        super().__init__()
        self.layers = nn.Sequential(
            nn.Linear(384, 128),
            nn.ReLU(),
            nn.Linear(128, 1)
        )

    def forward(self, x):
        return self.layers(x)

class AtomicNetNarrowing(nn.Module):
    def __init__(self):
        super().__init__()
        self.layers = nn.Sequential(
            nn.Linear(384, 128),
            nn.ReLU(),
            nn.Linear(128, 64),
            nn.ReLU(),
            nn.Linear(64, 1)
        )

    def forward(self, x):
        return self.layers(x)

class AtomicNetBigger(nn.Module):
    def __init__(self):
        super().__init__()
        self.layers = nn.Sequential(
            nn.Linear(384, 256),
            nn.ReLU(),
            nn.Linear(256, 128),
            nn.ReLU(),
            nn.Linear(128, 1)
        )

    def forward(self, x):
        return self.layers(x)
```

```
class AtomicNetPaper(nn.Module):
    def __init__(self):
        super().__init__()
        self.layers = nn.Sequential(
            nn.Linear(384, 128),
            nn.ReLU(),
            nn.Linear(128, 128),
            nn.ReLU(),
            nn.Linear(128, 64),
            nn.ReLU(),
            nn.Linear(64, 1)
        )
```

```
    def forward(self, x):
        return self.layers(x)
```

```
class AtomicNetPaperWide(nn.Module):
    def __init__(self):
        super().__init__()
        self.layers = nn.Sequential(
            nn.Linear(384, 256),
            nn.ReLU(),
            nn.Linear(256, 256),
            nn.ReLU(),
            nn.Linear(256, 128),
            nn.ReLU(),
            nn.Linear(128, 1)
        )
```

```
    def forward(self, x):
        return self.layers(x)
```

```
class AtomicNetDropout(nn.Module):
    def __init__(self):
        super().__init__()
        self.layers = nn.Sequential(
            nn.Linear(384, 128),
            nn.ReLU(),
            nn.Dropout(p=0.1),
            nn.Linear(128, 64),
            nn.ReLU(),
            nn.Dropout(p=0.1),
            nn.Linear(64, 1)
        )
```

```

    )

    def forward(self, x):
        return self.layers(x)

net_H = AtomicNet()
net_C = AtomicNet()
net_N = AtomicNet()
net_O = AtomicNet()

# ANI model requires a network for each atom type
# use torch.ANIModel() to compile atomic networks
ani_net = torchani.ANIModel([net_H, net_C, net_N, net_O])
model = nn.Sequential(
    aev_computer,
    ani_net
).to(device)

```

```

In [12]: train_data_batch = next(iter(train_data_loader))

loss_func = nn.MSELoss()
species = train_data_batch['species'].to(device)
coords = train_data_batch['coordinates'].to(device)
true_energies = train_data_batch['energies'].to(device).float()
_, pred_energies = model((species, coords))
loss = loss_func(true_energies, pred_energies)
print(loss)

```

tensor(0.0162, device='cuda:0', grad_fn=<MseLossBackward0>)

```

In [13]: class ANITrainer:
    def __init__(self, model, batch_size, learning_rate, epoch, l2):
        self.model = model

        num_params = sum(item.numel() for item in model.parameters())
        print(f"{model.__class__.__name__} - Number of parameters: {num_params}")

        self.batch_size = batch_size
        self.optimizer = torch.optim.Adam(model.parameters(), learning_rate, weight_decay=l2)
        self.epoch = epoch

    def train(self, train_data, val_data, early_stop=True, draw_curve=True):
        self.model.train()

        # init data loader

```

```

print("Initialize training data...")
train_data_loader = train_data.collate(self.batch_size).cache()

# definition of loss function: MSE is a good choice!
loss_func = nn.MSELoss()

# record epoch losses
train_loss_list = []
val_loss_list = []
lowest_val_loss = np.inf

for i in tqdm(range(self.epoch), leave=True):
    train_epoch_loss = 0.0
    for train_data_batch in train_data_loader:

        # compute energies
        species = train_data_batch['species'].to(device)
        coords = train_data_batch['coordinates'].to(device)
        true_energies = train_data_batch['energies'].to(device).float()
        _, pred_energies = self.model((species, coords))

        # compute loss
        batch_loss = loss_func(true_energies, pred_energies)

        # do a step
        self.optimizer.zero_grad() #clears previous batch gradient
        batch_loss.backward() #backprop
        self.optimizer.step()

        batch_importance = len(species) / len(train_data)
        train_epoch_loss += batch_loss.item() * batch_importance

    # use the self.evaluate to get loss on the validation set
    val_epoch_loss = self.evaluate(val_data)

    # append the losses
    train_loss_list.append(train_epoch_loss)
    val_loss_list.append(val_epoch_loss)

    if early_stop:
        if val_epoch_loss < lowest_val_loss:
            lowest_val_loss = val_epoch_loss
            weights = self.model.state_dict()

if draw_curve:

```

```

fig, ax = plt.subplots(1, 1, figsize=(5, 4), constrained_layout=True)
ax.set_yscale("log")
# Plot train loss and validation loss
ax.plot(np.arange(self.epoch), train_loss_list, label='Train')
ax.plot(np.arange(self.epoch), val_loss_list, label='Validation')
ax.legend()
ax.set_xlabel("# Batch")
ax.set_ylabel("Loss")

if early_stop:
    self.model.load_state_dict(weights)

return train_loss_list, val_loss_list

def evaluate(self, data, draw_plot=False):

    # init data loader
    data_loader = data.collate(self.batch_size).cache()

    # init loss function
    loss_func = nn.MSELoss()
    total_loss = 0.0

    if draw_plot:
        true_energies_all = []
        pred_energies_all = []

    with torch.no_grad():
        for batch_data in data_loader:

            # compute energies
            species = batch_data['species'].to(device)
            coords = batch_data['coordinates'].to(device)
            true_energies = batch_data['energies'].to(device).float()
            _, pred_energies = self.model((species, coords))

            # compute loss
            batch_loss = loss_func(true_energies, pred_energies)

            batch_importance = len(species) / len(data)
            total_loss += batch_loss.item() * batch_importance

            if draw_plot:
                true_energies_all.append(true_energies.detach().cpu().numpy().flatten())

```

```

        pred_energies_all.append(pred_energies.detach().cpu().numpy().flatten())

    if draw_plot:
        true_energies_all = np.concatenate(true_energies_all)
        pred_energies_all = np.concatenate(pred_energies_all)
        # Report the mean absolute error
        # The unit of energies in the dataset is hartree
        # please convert it to kcal/mol when reporting the mean absolute error
        # 1 hartree = 627.5094738898777 kcal/mol
        # MAE = mean(|true - pred|)
        hartree2kcalmol = 627.5094738898777
        mae = np.mean(np.abs(true_energies_all - pred_energies_all)) * hartree2kcalmol
        fig, ax = plt.subplots(1, 1, figsize=(5, 4), constrained_layout=True)
        ax.scatter(true_energies_all, pred_energies_all, label=f"MAE: {mae:.2f} kcal/mol", s=2)
        ax.set_xlabel("Ground Truth")
        ax.set_ylabel("Predicted")
        xmin, xmax = ax.get_xlim()
        ymin, ymax = ax.get_ylim()
        vmin, vmax = min(xmin, ymin), max(xmax, ymax)
        ax.set_xlim(vmin, vmax)
        ax.set_ylim(vmin, vmax)
        ax.plot([vmin, vmax], [vmin, vmax], color='red')
        ax.legend()

    return total_loss

```

```

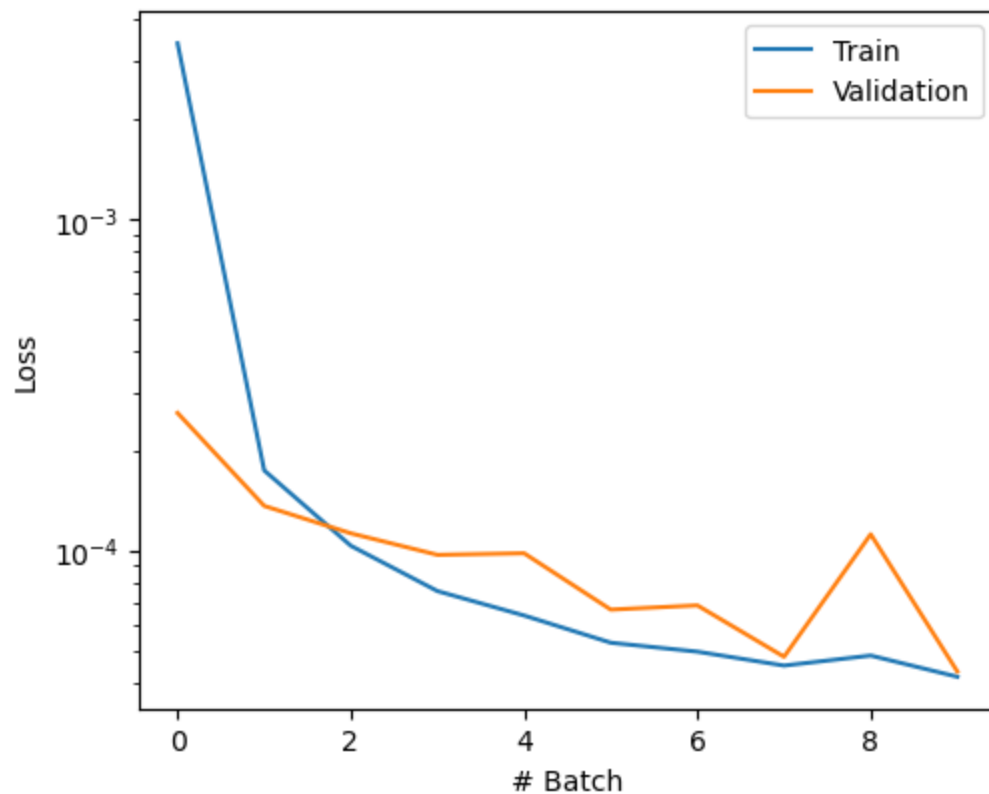
In [10]: small_train = train_data.split(0.05)[0]
        small_val = val_data.split(0.1)[0]
        trainer = ANITrainer(model, batch_size=256, learning_rate=1e-3, epoch=10, l2=1e-5)
        train_losses, val_losses = trainer.train(small_train, small_val)

```

Sequential - Number of parameters: 197636

Initialize training data...

100%|██████████| 10/10 [00:18<00:00, 1.80s/it]



```
In [22]: #20% data on basic model, first will compare differing layer architectures
exp_train = train_data.split(0.20)[0]
exp_val = val_data.split(0.1)[0]

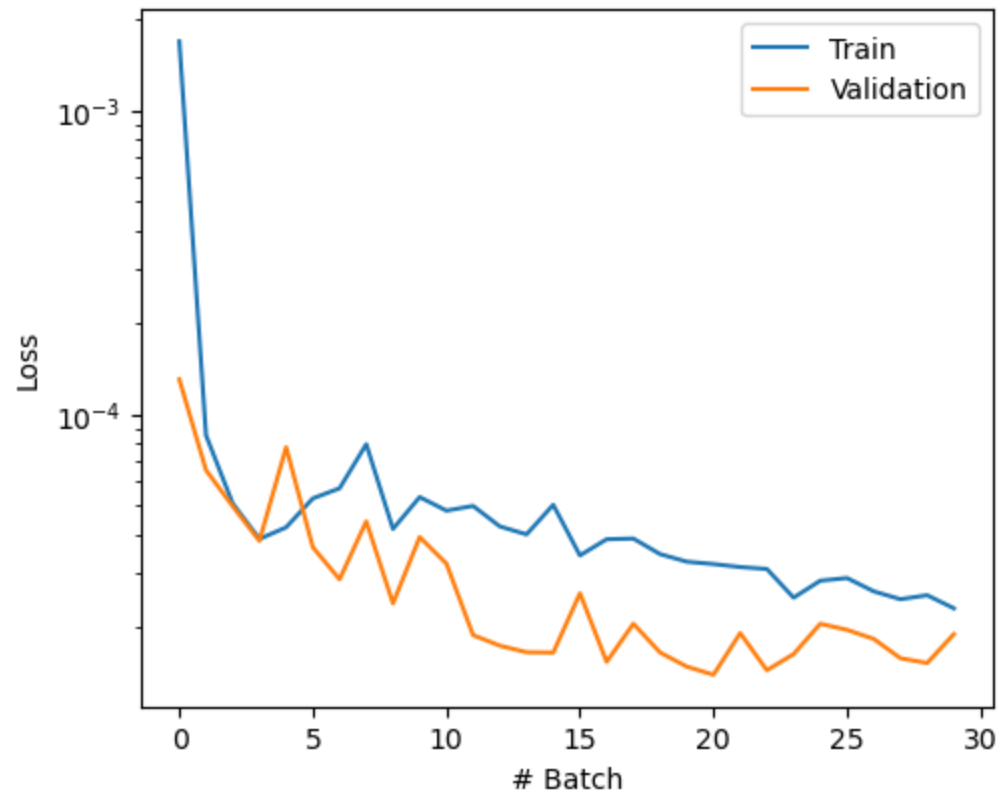
ani_net = torchani.ANIModel([AtomicNet(), AtomicNet(), AtomicNet(), AtomicNet()])
model = nn.Sequential(aev_computer, ani_net).to(device)

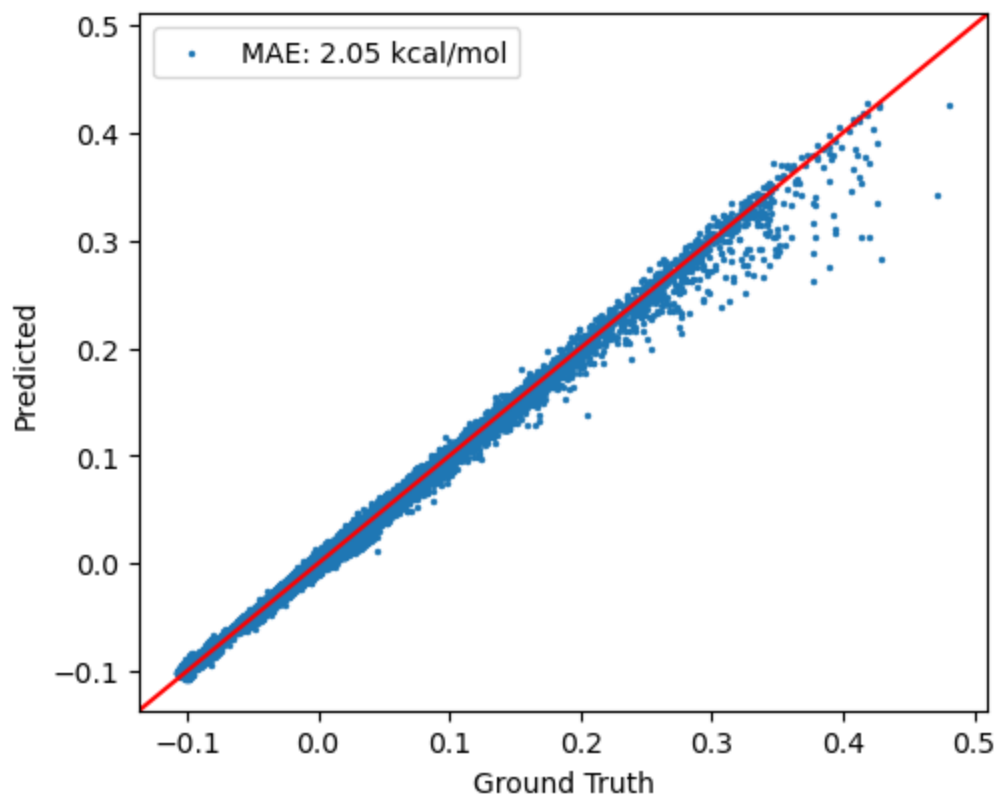
trainer = ANITrainer(model, batch_size=512, learning_rate=1e-3, epoch=30, l2=1e-5)
train_losses, val_losses = trainer.train(exp_train, exp_val)
test_loss = trainer.evaluate(test_data, draw_plot=True)
print(f"20% data test loss: {test_loss:.6f}")
#trial 1: 20% data test loss: 0.000019
#trial 2: 20% data test loss: 0.000020
#trial 3: 20% data test loss: 0.000015
#trial 4: 20% data test loss: 0.000014
#trial 5: 20% data test loss: 0.000019
```

Sequential – Number of parameters: 197636
Initialize training data...

100%|██████████| 30/30 [01:06<00:00, 2.20s/it]

20% data test loss: 0.000019





```
In [23]: exp_train = train_data.split(0.20)[0]
exp_val = val_data.split(0.1)[0]

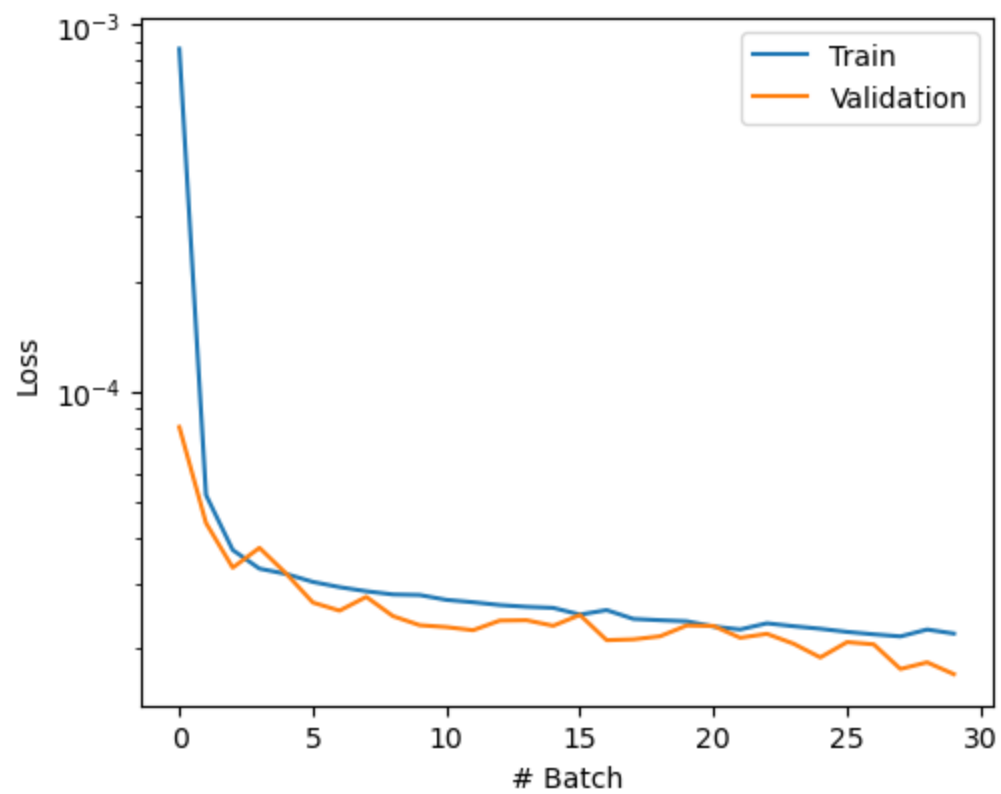
ani_net = torchani.ANIModel([AtomicNetNarrowing(), AtomicNetNarrowing(), AtomicNetNarrowing(), AtomicNetNarrowing()])
model = nn.Sequential(aev_computer, ani_net).to(device)

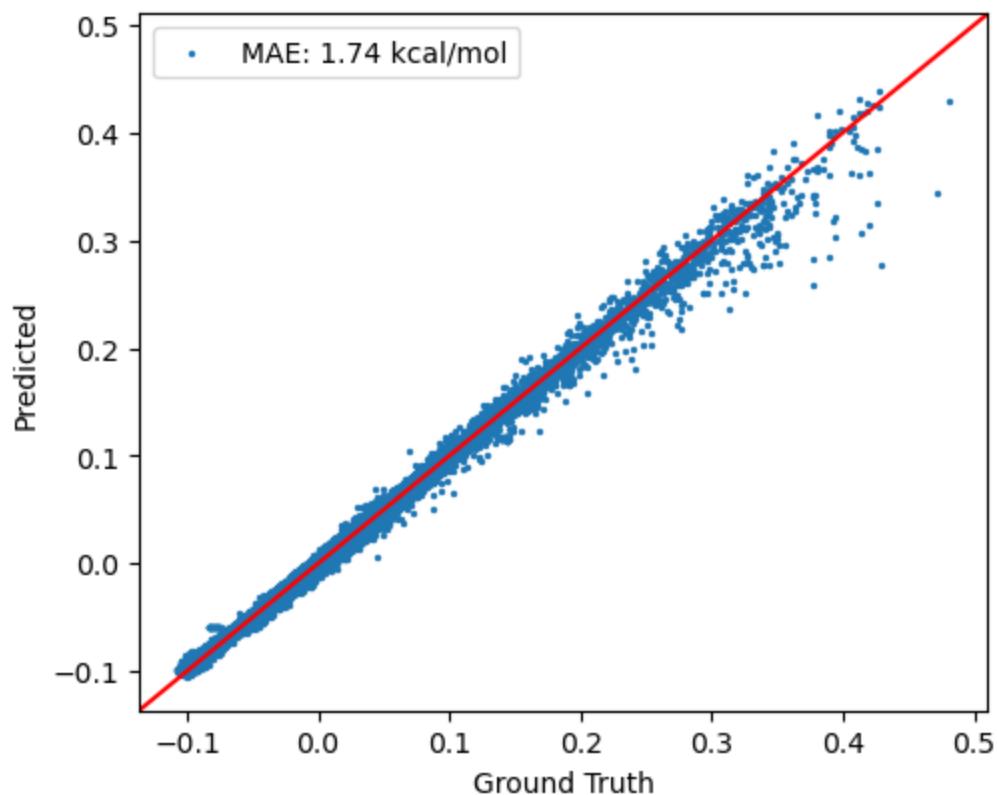
trainer = ANITrainer(model, batch_size=512, learning_rate=1e-3, epoch=30, l2=1e-5)
train_losses, val_losses = trainer.train(exp_train, exp_val)
test_loss = trainer.evaluate(test_data, draw_plot=True)
print(f"Narrowing network test loss: {test_loss:.6f}")
#trial 1: Narrowing network test loss: 0.000023
#trial 2: Narrowing Network test loss: 0.000018
#trial 3: Narrowing network test loss: 0.000016
#trial 4: Narrowing network test loss: 0.000016
#trail 5: Narrowing network test loss: 0.000017
```

Sequential – Number of parameters: 230404
Initialize training data...

100%|██████████| 30/30 [01:07<00:00, 2.25s/it]

Narrowing network test loss: 0.000017





```
In [24]: exp_train = train_data.split(0.20)[0]
exp_val = val_data.split(0.1)[0]

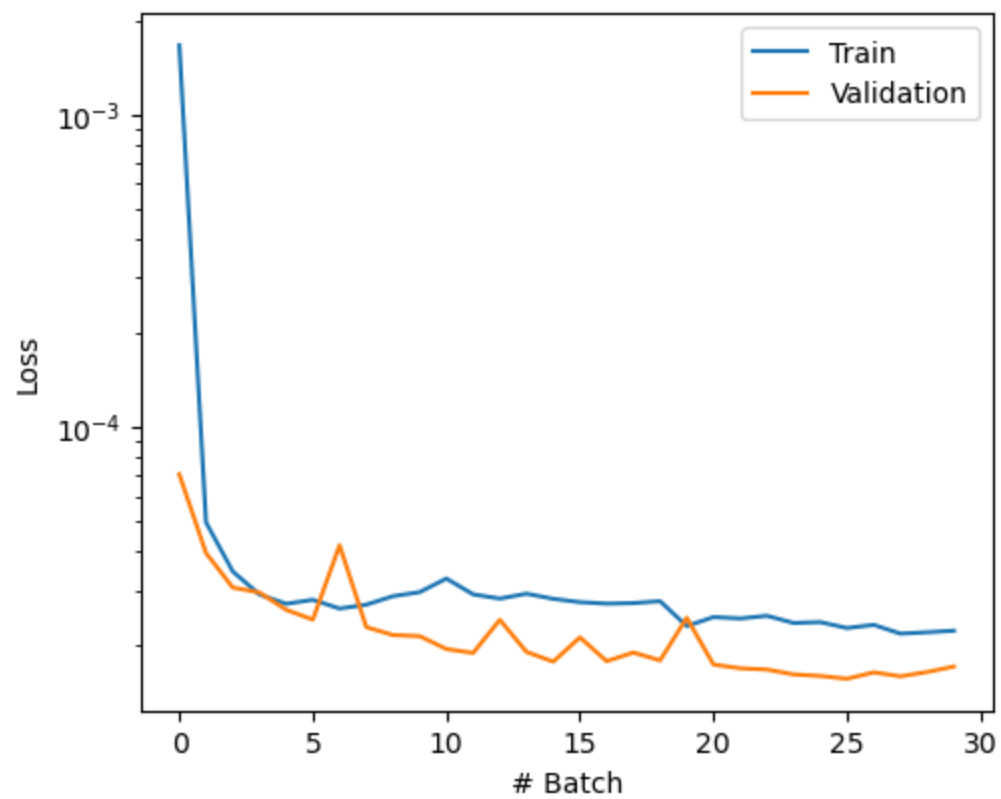
ani_net = torchani.ANIModel([AtomicNetBigger(), AtomicNetBigger(), AtomicNetBigger(), AtomicNetBigger()])
model = nn.Sequential(aev_computer, ani_net).to(device)

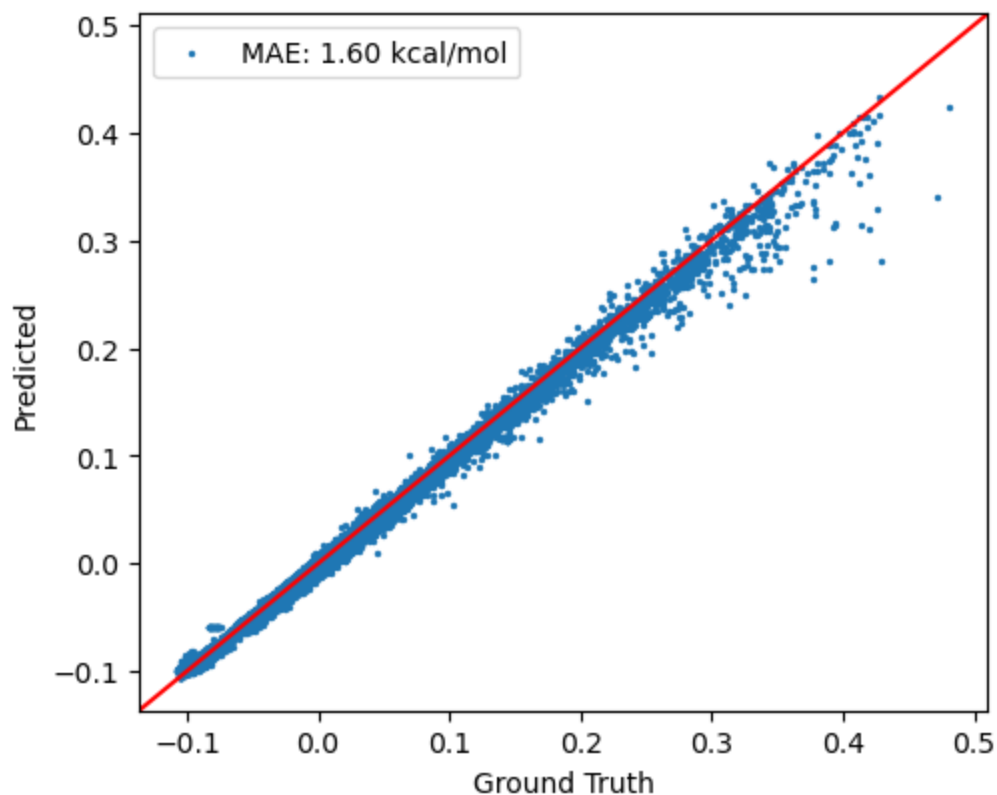
trainer = ANITrainer(model, batch_size=512, learning_rate=1e-3, epoch=30, l2=1e-5)
train_losses, val_losses = trainer.train(exp_train, exp_val)
test_loss = trainer.evaluate(test_data, draw_plot=True)
print(f"Bigger Network test loss: {test_loss:.6f}")
#trial 1: Bigger Network test loss: 0.000020
#trial 2: Bigger Network test loss: 0.000018
#trial 3: Bigger Network test loss: 0.000019
#trial 4: Bigger Network test loss: 0.000015
#trail 5: Bigger Network test loss: 0.000017
```

Sequential – Number of parameters: 526340
Initialize training data...

100%|██████████| 30/30 [01:12<00:00, 2.40s/it]

Bigger Network test loss: 0.000017





```
In [25]: exp_train = train_data.split(0.20)[0]
exp_val = val_data.split(0.1)[0]

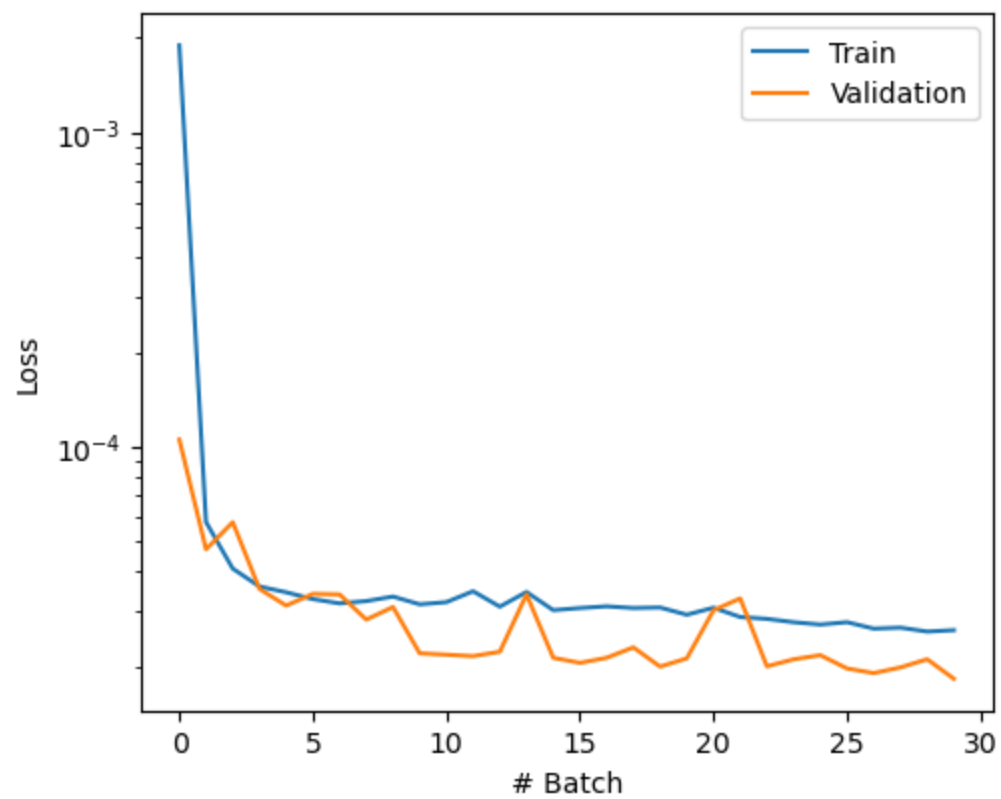
ani_net = torchani.ANIModel([AtomicNetPaper(), AtomicNetPaper(), AtomicNetPaper(), AtomicNetPaper()])
model = nn.Sequential(aev_computer, ani_net).to(device)

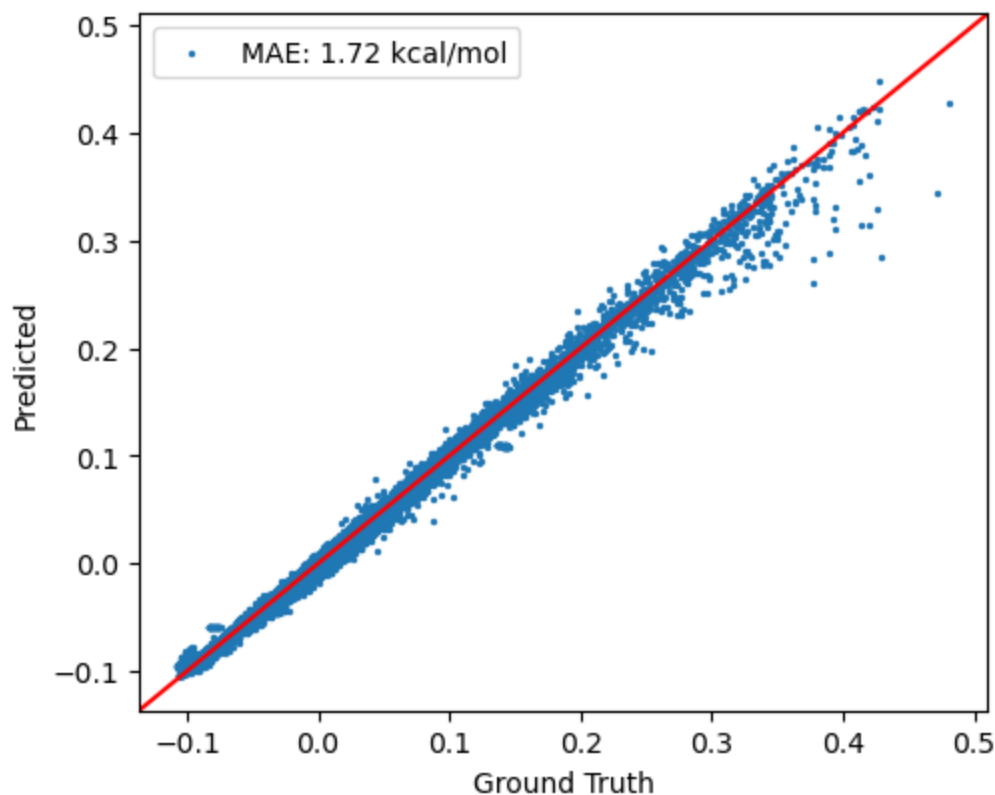
trainer = ANITrainer(model, batch_size=512, learning_rate=1e-3, epoch=30, l2=1e-5)
train_losses, val_losses = trainer.train(exp_train, exp_val)
test_loss = trainer.evaluate(test_data, draw_plot=True)
print(f"Paper Network test loss: {test_loss:.6f}")
#trial 1: Paper Network test loss: 0.000018
#trial 2: Paper Network test loss: 0.000018
#trial 3: Paper Network test loss: 0.000019
#trial 4: Paper Network test loss: 0.000017
#trail 5: Paper Network test loss: 0.000018
```

Sequential – Number of parameters: 296452
Initialize training data...

100%|██████████| 30/30 [01:14<00:00, 2.48s/it]

Paper Network test loss: 0.000018





In [26]: the paper similar architecture **with** two hidden layers has low loss but most importantly it has the lowest standard deviation

Cell In[26], line 1

the paper similar architecture with two hidden layers has the lowest loss for a similar number of parameters as the other low loss architectures so I will be using this going forward

SyntaxError: invalid syntax. Perhaps you forgot a comma?

```
In [ ]: exp_train = train_data.split(0.20)[0]
exp_val = val_data.split(0.1)[0]

ani_net = torchani.ANIModel([AtomicNetPaperWide(), AtomicNetPaperWide(), AtomicNetPaperWide(), AtomicNetPaperWide()])
model = nn.Sequential(aev_computer, ani_net).to(device)

trainer = ANITrainer(model, batch_size=512, learning_rate=1e-3, epoch=30, l2=1e-5)
train_losses, val_losses = trainer.train(exp_train, exp_val)
test_loss = trainer.evaluate(test_data, draw_plot=True)
print(f"Paper Network wide test loss: {test_loss:.6f}")
#trial 1: Paper Network wide test loss: 0.000023
#trial 2: Paper Network wide test loss: 0.000021
```

```
In [35]: exp_train = train_data.split(0.50)[0]
exp_val = val_data.split(0.1)[0]

ani_net = torchani.ANIModel([AtomicNetPaper(), AtomicNetPaper(), AtomicNetPaper(), AtomicNetPaper()])
model = nn.Sequential(aev_computer, ani_net).to(device)

trainer = ANITrainer(model, batch_size=512, learning_rate=1e-3, epoch=30, l2=1e-5)
train_losses, val_losses = trainer.train(exp_train, exp_val)
test_loss = trainer.evaluate(test_data, draw_plot=True)
print(f"Paper Network 50% train data test loss: {test_loss:.6f}")

print(f"Final train loss: {train_losses[-1]:.6f}")
print(f"Final val loss: {val_losses[-1]:.6f}")
print(f"Train/Val gap: {abs(train_losses[-1] - val_losses[-1]):.6f}")

best_epoch = val_losses.index(min(val_losses))
print(f"Best epoch: {best_epoch}/{len(val_losses)}")
print(f"Best val loss: {min(val_losses):.6f}")

improvement = (train_losses[0] - train_losses[-1]) / train_losses[0] * 100
print(f"Train loss improvement: {improvement:.1f}%")
```

Sequential – Number of parameters: 296452

Initialize training data...

100%|██████████| 30/30 [02:47<00:00, 5.59s/it]

Paper Network 50% train data test loss: 0.000018

Final train loss: 0.000021

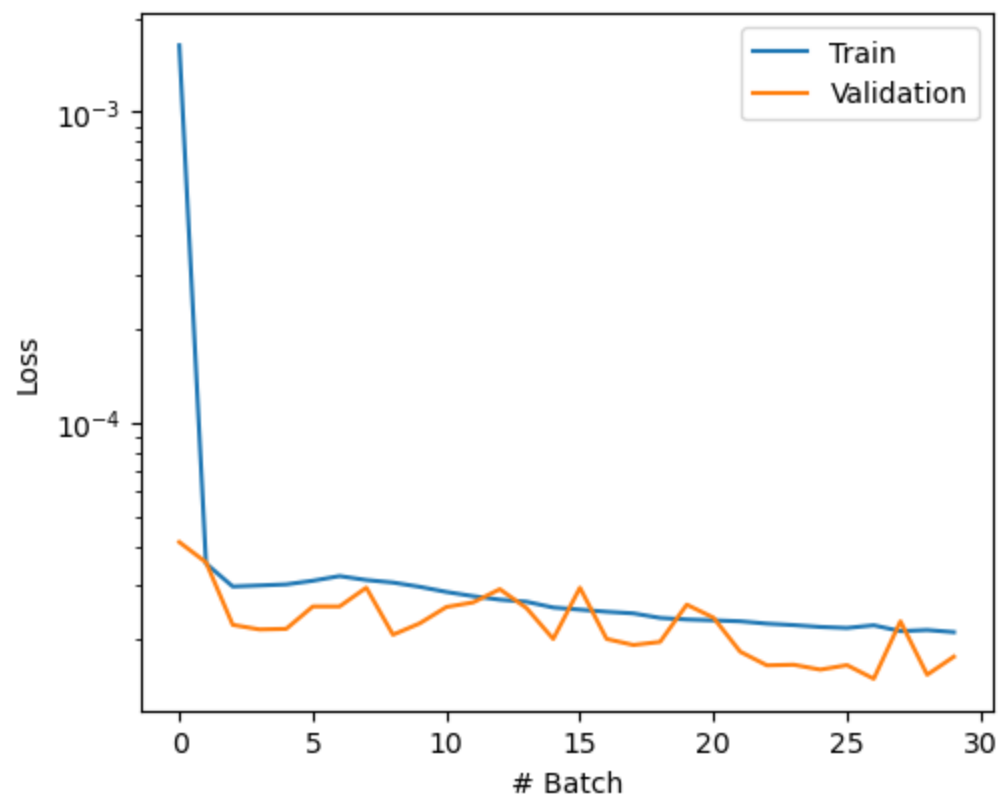
Final val loss: 0.000018

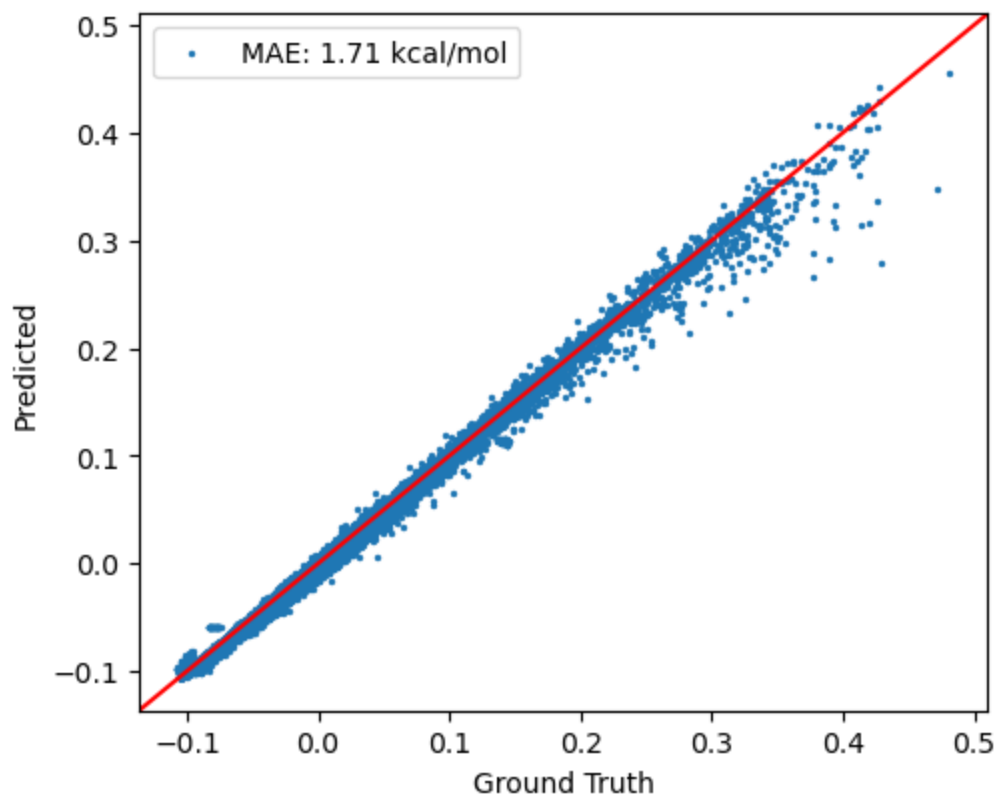
Train/Val gap: 0.000004

Best epoch: 26/30

Best val loss: 0.000015

Train loss improvement: 98.7%





t1: Paper Network 50% train data test loss: 0.000016 t2: Paper Network 50% train data test loss: 0.000016 t3: Paper Network 50% train data test loss: 0.000016, Final train loss: 0.000021, Final val loss: 0.000016, Train/Val gap: 0.000005, Best epoch: 29/30, Best val loss: 0.000016, Train loss improvement: 98.8% t4: Paper Network 50% train data test loss: 0.000016 Final train loss: 0.000023 Final val loss: 0.000017 Train/Val gap: 0.000006 Best epoch: 29/30 Best val loss: 0.000017 Train loss improvement: 98.3% t5: Paper Network 50% train data test loss: 0.000018 Final train loss: 0.000021 Final val loss: 0.000018 Train/Val gap: 0.000004 Best epoch: 26/30 Best val loss: 0.000015 Train loss improvement: 98.7%

```
In [ ]: exp_train = train_data.split(0.80)[0]
exp_val = val_data.split(0.1)[0]

ani_net = torchani.ANIModel([AtomicNetPaper(), AtomicNetPaper(), AtomicNetPaper(), AtomicNetPaper()])
model = nn.Sequential(aev_computer, ani_net).to(device)

trainer = ANITrainer(model, batch_size=512, learning_rate=1e-3, epoch=30, l2=1e-5)
train_losses, val_losses = trainer.train(exp_train, exp_val)
test_loss = trainer.evaluate(test_data, draw_plot=True)
print(f"Paper Network 80% train data test loss: {test_loss:.6f}")
```

```

In [34]: exp_train = train_data.split(0.6)[0]
exp_val = val_data.split(0.1)[0]

ani_net = torchani.ANIModel([AtomicNetPaper(), AtomicNetPaper(), AtomicNetPaper(), AtomicNetPaper()])
model = nn.Sequential(aev_computer, ani_net).to(device)

trainer = ANITrainer(model, batch_size=512, learning_rate=1e-3, epoch=30, l2=1e-5)
train_losses, val_losses = trainer.train(exp_train, exp_val)
test_loss = trainer.evaluate(test_data, draw_plot=True)
print(f"Paper Network 60% train data test loss: {test_loss:.6f}")

print(f"Final train loss: {train_losses[-1]:.6f}")
print(f"Final val loss: {val_losses[-1]:.6f}")
print(f"Train/Val gap: {abs(train_losses[-1] - val_losses[-1]):.6f}")

best_epoch = val_losses.index(min(val_losses))
print(f"Best epoch: {best_epoch}/{len(val_losses)}")
print(f"Best val loss: {min(val_losses):.6f}")

improvement = (train_losses[0] - train_losses[-1]) / train_losses[0] * 100
print(f"Train loss improvement: {improvement:.1f}%")

```

Sequential – Number of parameters: 296452

Initialize training data...

0%| | 0/30 [00:04<?, ?it/s]

KeyboardInterrupt

Traceback (most recent call last)

Cell In[34], line 8

```
5 model = nn.Sequential(aev_computer, ani_net).to(device)
7 trainer = ANITrainer(model, batch_size=512, learning_rate=1e-3, epoch=30, l2=1e-5)
----> 8 train_losses, val_losses = trainer.train(exp_train, exp_val)
9 test_loss = trainer.evaluate(test_data, draw_plot=True)
10 print(f"Paper Network 60% train data test loss: {test_loss:.6f}")
```

Cell In[9], line 35, in ANITrainer.train(self, train_data, val_data, early_stop, draw_curve)

```
33 coords = train_data_batch['coordinates'].to(device)
34 true_energies = train_data_batch['energies'].to(device).float()
----> 35 _, pred_energies = self.model((species, coords))
37 # compute loss
38 batch_loss = loss_func(true_energies, pred_energies)
```

File /global/scratch/users/ericwangyz/chem242/ani/lib/python3.10/site-packages/torch/nn/modules/module.py:1511, in Module._wrapped_call_impl(self, *args, **kwargs)

```
1509     return self._compiled_call_impl(*args, **kwargs) # type: ignore[misc]
1510 else:
-> 1511     return self._call_impl(*args, **kwargs)
```

File /global/scratch/users/ericwangyz/chem242/ani/lib/python3.10/site-packages/torch/nn/modules/module.py:1520, in Module._call_impl(self, *args, **kwargs)

```
1515 # If we don't have any hooks, we want to skip the rest of the logic in
1516 # this function, and just call forward.
1517 if not (self._backward_hooks or self._backward_pre_hooks or self._forward_hooks or self._forward_pre_hooks
1518         or _global_backward_pre_hooks or _global_backward_hooks
1519         or _global_forward_hooks or _global_forward_pre_hooks):
-> 1520     return forward_call(*args, **kwargs)
1522 try:
1523     result = None
```

File /global/scratch/users/ericwangyz/chem242/ani/lib/python3.10/site-packages/torch/nn/modules/container.py:217, in Sequential.forward(self, input)

```
215 def forward(self, input):
216     for module in self:
--> 217         input = module(input)
218     return input
```

File /global/scratch/users/ericwangyz/chem242/ani/lib/python3.10/site-packages/torch/nn/modules/module.py:1511, in Module._wrapped_call_impl(self, *args, **kwargs)

```
1509     return self._compiled_call_impl(*args, **kwargs) # type: ignore[misc]
1510 else:
-> 1511     return self._call_impl(*args, **kwargs)
```

```

File /global/scratch/users/ericwangyz/chem242/ani/lib/python3.10/site-packages/torch/nn/modules/module.py:1520, in Module._call_impl(self, *args, **kwargs)
    1515 # If we don't have any hooks, we want to skip the rest of the logic in
    1516 # this function, and just call forward.
    1517 if not (self._backward_hooks or self._backward_pre_hooks or self._forward_hooks or self._forward_pre_hooks
    1518         or _global_backward_pre_hooks or _global_backward_hooks
    1519         or _global_forward_hooks or _global_forward_pre_hooks):
-> 1520     return forward_call(*args, **kwargs)
    1522 try:
    1523     result = None

```

```

File /global/scratch/users/ericwangyz/chem242/ani/lib/python3.10/site-packages/torchani/nn.py:60, in ANIModel.forward(self, species_aev, cell, pbc)
    57 species, aev = species_aev
    58 assert species.shape == aev.shape[:-1]
----> 60 atomic_energies = self._atomic_energies((species, aev))
    61 # shape of atomic energies is (C, A)
    62 return SpeciesEnergies(species, torch.sum(atomic_energies, dim=1))

```

```

File /global/scratch/users/ericwangyz/chem242/ani/lib/python3.10/site-packages/torchani/nn.py:79, in ANIModel._atomic_energies(self, species_aev)
    77 if midx.shape[0] > 0:
    78     input_ = aev.index_select(0, midx)
----> 79     output.masked_scatter_(mask, m(input_).flatten())
    80 output = output.view_as(species)
    81 return output

```

KeyboardInterrupt:

t1: Paper Network 60% train data test loss: 0.000014, Final train loss: 0.000021, Final val loss: 0.000014, Train/Val gap: 0.000006, Best epoch: 26/30, Best val loss: 0.000014, Train loss improvement: 92.2% t2: Paper Network 60% train data test loss: 0.000014, Final train loss: 0.000021, Final val loss: 0.000014, Train/Val gap: 0.000007, Best epoch: 27/30, Best val loss: 0.000014, Train loss improvement: 96.8% t3: Paper Network 60% train data test loss: 0.000018 Final train loss: 0.000021 Final val loss: 0.000018 Train/Val gap: 0.000003 Best epoch: 27/30 Best val loss: 0.000015 Train loss improvement: 94.4% t4: Paper Network 60% train data test loss: 0.000016 Final train loss: 0.000022 Final val loss: 0.000016 Train/Val gap: 0.000007 Best epoch: 27/30 Best val loss: 0.000016 Train loss improvement: 98.9%

```

In [ ]: exp_train = train_data.split(0.4)[0]
exp_val = val_data.split(0.1)[0]

ani_net = torchani.ANIModel([AtomicNetPaper(), AtomicNetPaper(), AtomicNetPaper(), AtomicNetPaper()])
model = nn.Sequential(aev_computer, ani_net).to(device)

```

```

trainer = ANITrainer(model, batch_size=512, learning_rate=1e-3, epoch=30, l2=1e-5)
train_losses, val_losses = trainer.train(exp_train, exp_val)
test_loss = trainer.evaluate(test_data, draw_plot=True)
print(f"Paper Network 40% train data test loss: {test_loss:.6f}")
#Paper Network 40% train data test loss: 0.000037
#Paper Network 40% train data test loss: 0.000022

```

```

In [37]: exp_train = train_data.split(0.7)[0]
exp_val = val_data.split(0.1)[0]

ani_net = torchani.ANIModel([AtomicNetPaper(), AtomicNetPaper(), AtomicNetPaper(), AtomicNetPaper()])
model = nn.Sequential(aev_computer, ani_net).to(device)

trainer = ANITrainer(model, batch_size=512, learning_rate=1e-3, epoch=30, l2=1e-5)
train_losses, val_losses = trainer.train(exp_train, exp_val)
test_loss = trainer.evaluate(test_data, draw_plot=True)
print(f"Paper Network 70% train data test loss: {test_loss:.6f}")

print(f"Final train loss: {train_losses[-1]:.6f}")
print(f"Final val loss: {val_losses[-1]:.6f}")
print(f"Train/Val gap: {abs(train_losses[-1] - val_losses[-1]):.6f}")

best_epoch = val_losses.index(min(val_losses))
print(f"Best epoch: {best_epoch}/{len(val_losses)}")
print(f"Best val loss: {min(val_losses):.6f}")

improvement = (train_losses[0] - train_losses[-1]) / train_losses[0] * 100
print(f"Train loss improvement: {improvement:.1f}%")

```

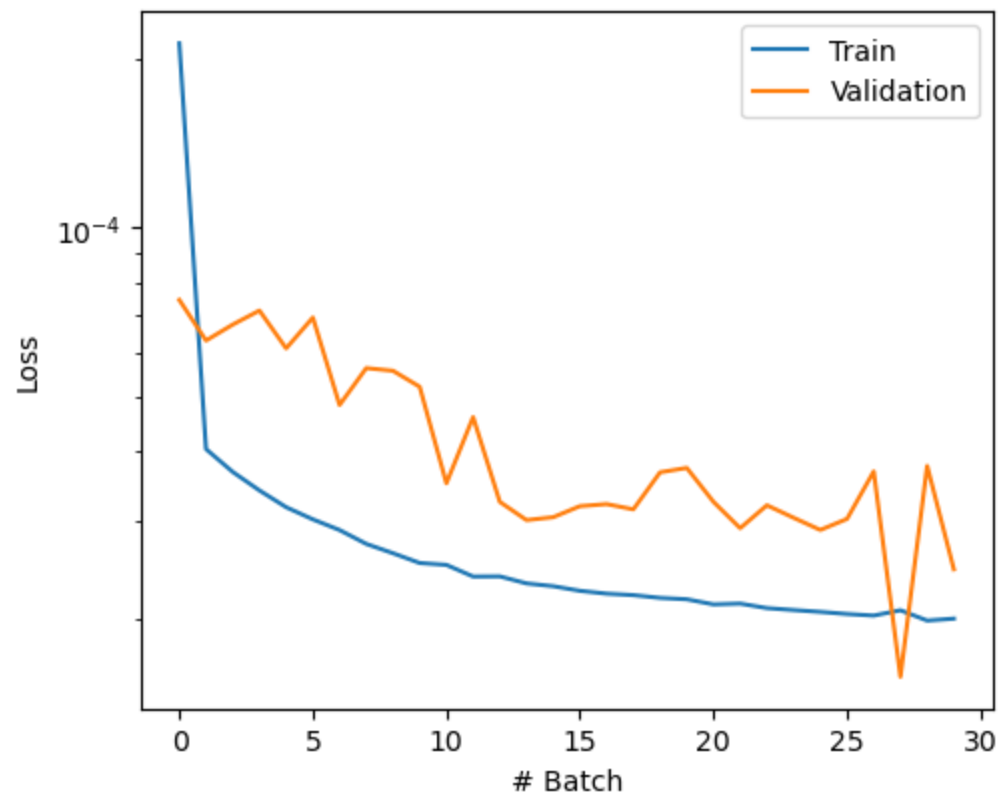
Sequential – Number of parameters: 296452

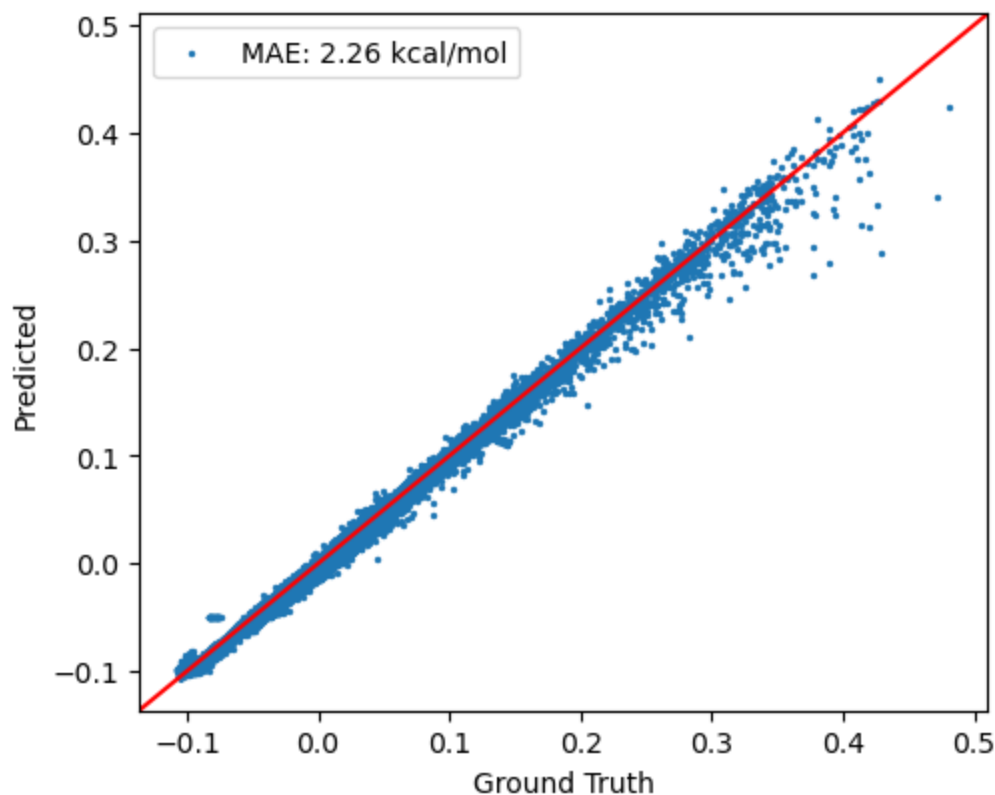
Initialize training data...

```

100%|██████████| 30/30 [04:03<00:00, 8.12s/it]
Paper Network 60% train data test loss: 0.000024
Final train loss: 0.000020
Final val loss: 0.000025
Train/Val gap: 0.000005
Best epoch: 27/30
Best val loss: 0.000016
Train loss improvement: 90.6%

```



t1: Paper Network 70% train data test loss: 0.000022 Final train loss: 0.000020 Final val loss: 0.000022 Train/Val gap: 0.000002 Best epoch: 28/30 Best val loss: 0.000017 Train loss improvement: 93.5% t2: Paper Network 70% train data test loss: 0.000024 Final train loss: 0.000020 Final val loss: 0.000025 Train/Val gap: 0.000005 Best epoch: 27/30 Best val loss: 0.000016 Train loss improvement: 90.6%

worsening performance compared to 60% so will stick with that

```
In [40]: #60% train data, 60 epoch trial
exp_train = train_data.split(0.6)[0]
exp_val = val_data.split(0.1)[0]

ani_net = torchani.ANIModel([AtomicNetPaper(), AtomicNetPaper(), AtomicNetPaper(), AtomicNetPaper()])
model = nn.Sequential(aev_computer, ani_net).to(device)

trainer = ANITrainer(model, batch_size=512, learning_rate=1e-3, epoch=60, l2=1e-5)
train_losses, val_losses = trainer.train(exp_train, exp_val)
test_loss = trainer.evaluate(test_data, draw_plot=True)
print(f"Paper Network 60% train data test loss: {test_loss:.6f}")

print(f"Final train loss: {train_losses[-1]:.6f}")
```

```

print(f"Final val loss: {val_losses[-1]:.6f}")
print(f"Train/Val gap: {abs(train_losses[-1] - val_losses[-1]):.6f}")

best_epoch = val_losses.index(min(val_losses))
print(f"Best epoch: {best_epoch}/{len(val_losses)}")
print(f"Best val loss: {min(val_losses):.6f}")

improvement = (train_losses[0] - train_losses[-1]) / train_losses[0] * 100
print(f"Train loss improvement: {improvement:.1f}%")

```

Sequential – Number of parameters: 296452

Initialize training data...

100%|██████████| 60/60 [06:37<00:00, 6.62s/it]

Paper Network 60% train data test loss: 0.000015

Final train loss: 0.000020

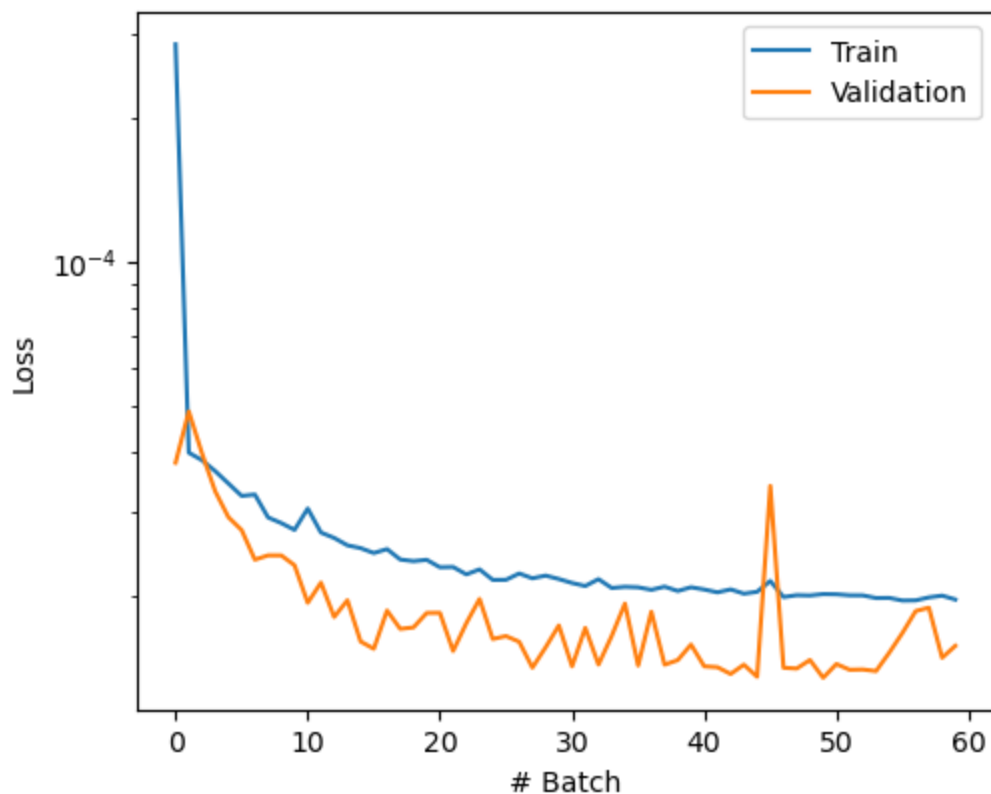
Final val loss: 0.000016

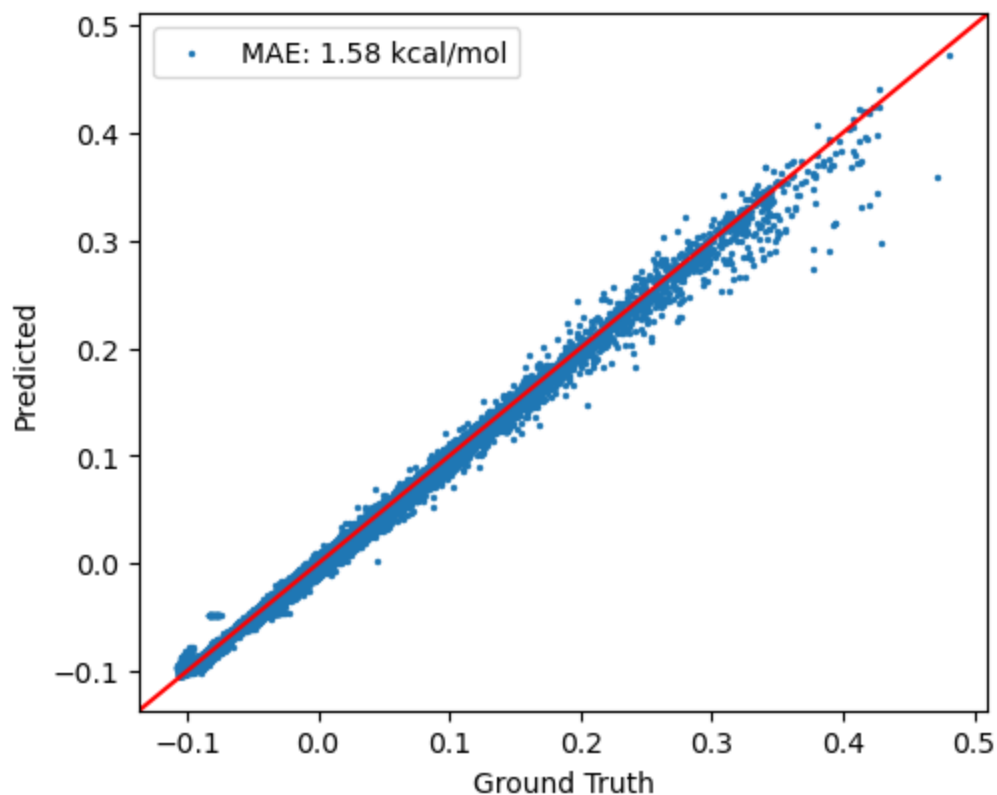
Train/Val gap: 0.000004

Best epoch: 49/60

Best val loss: 0.000014

Train loss improvement: 93.1%





t1: Paper Network 60% train data test loss: 0.000014 Final train loss: 0.000018 Final val loss: 0.000014 Train/Val gap: 0.000004 Best epoch: 57/60 Best val loss: 0.000013 Train loss improvement: 99.1% t2: Paper Network 60% train data test loss: 0.000015 Final train loss: 0.000022 Final val loss: 0.000015 Train/Val gap: 0.000007 Best epoch: 57/60 Best val loss: 0.000015 Train loss improvement: 99.0% t3: Paper Network 60% train data test loss: 0.000015 Final train loss: 0.000020 Final val loss: 0.000016 Train/Val gap: 0.000004 Best epoch: 49/60 Best val loss: 0.000014 Train loss improvement: 93.1%

```
In [15]: #60% train data, 90 epoch trial
exp_train = train_data.split(0.6)[0]
exp_val = val_data.split(0.1)[0]

ani_net = torchani.ANIModel([AtomicNetPaper(), AtomicNetPaper(), AtomicNetPaper(), AtomicNetPaper()])
model = nn.Sequential(aev_computer, ani_net).to(device)

trainer = ANITrainer(model, batch_size=512, learning_rate=1e-3, epoch=90, l2=1e-5)
train_losses, val_losses = trainer.train(exp_train, exp_val)
test_loss = trainer.evaluate(test_data, draw_plot=True)
print(f"Paper Network 60% train 90 epoch test loss: {test_loss:.6f}")
```

```

print(f"Final train loss: {train_losses[-1]:.6f}")
print(f"Final val loss: {val_losses[-1]:.6f}")
print(f"Train/Val gap: {abs(train_losses[-1] - val_losses[-1]):.6f}")

best_epoch = val_losses.index(min(val_losses))
print(f"Best epoch: {best_epoch}/{len(val_losses)}")
print(f"Best val loss: {min(val_losses):.6f}")

improvement = (train_losses[0] - train_losses[-1]) / train_losses[0] * 100
print(f"Train loss improvement: {improvement:.1f}%")

```

Sequential – Number of parameters: 296452

Initialize training data...

100%|██████████| 90/90 [09:50<00:00, 6.56s/it]

Paper Network 60% train 90 epoch test loss: 0.000016

Final train loss: 0.000019

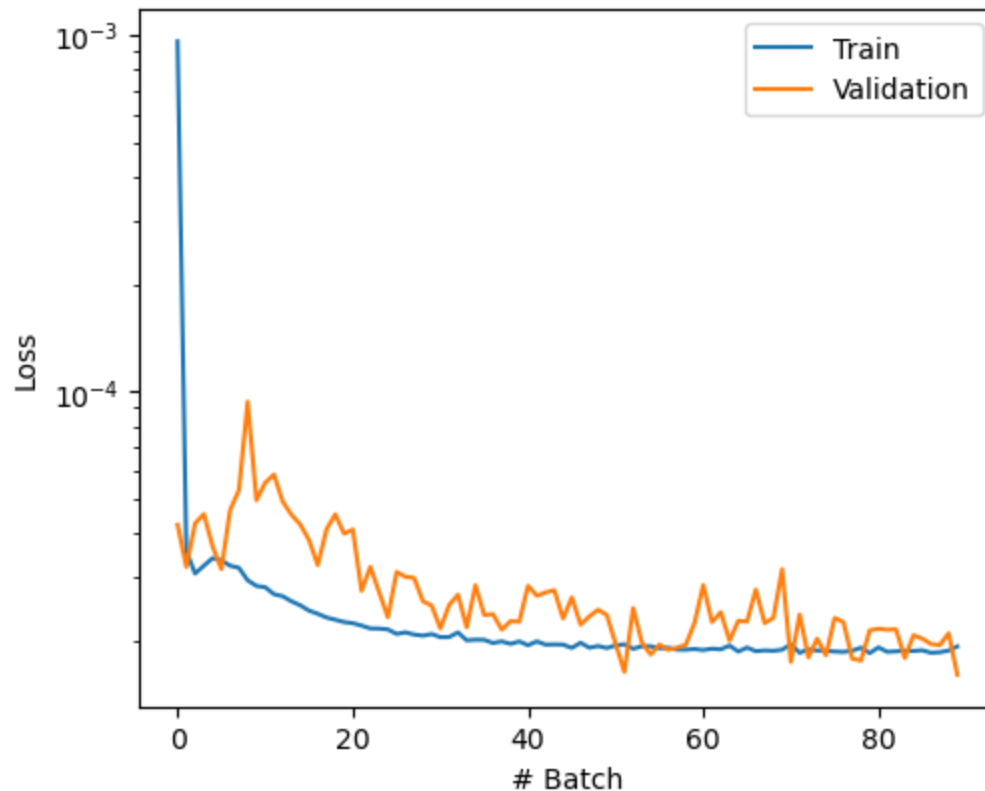
Final val loss: 0.000016

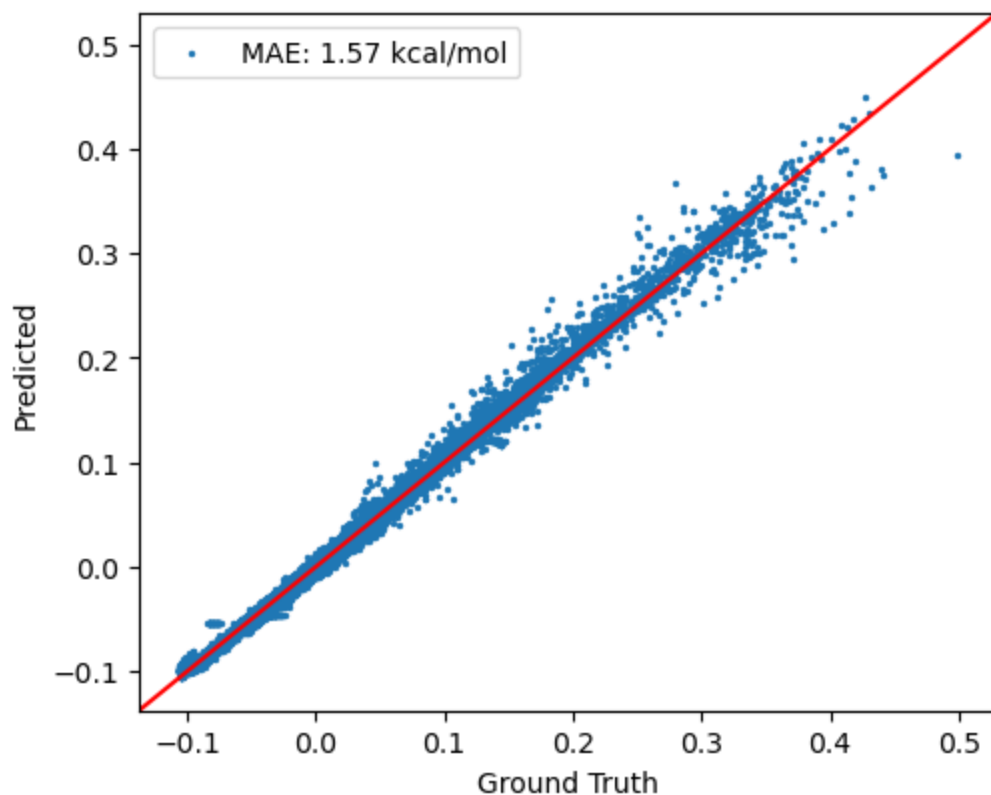
Train/Val gap: 0.000003

Best epoch: 89/90

Best val loss: 0.000016

Train loss improvement: 98.0%





t1: Paper Network 60% train 90 epoch test loss: 0.000014 Final train loss: 0.000018 Final val loss: 0.000014 Train/Val gap: 0.000004 Best epoch: 89/90 Best val loss: 0.000014 Train loss improvement: 99.0% t2: Paper Network 60% train 90 epoch test loss: 0.000031 Final train loss: 0.000020 Final val loss: 0.000031 Train/Val gap: 0.000011 Best epoch: 74/90 Best val loss: 0.000019 Train loss improvement: 98.8% t3: Paper Network 60% train 90 epoch test loss: 0.000016 Final train loss: 0.000019 Final val loss: 0.000016 Train/Val gap: 0.000003 Best epoch: 89/90 Best val loss: 0.000016 Train loss improvement: 98.0%

```
In [16]: #higher learning rate
exp_train = train_data.split(0.6)[0]
exp_val = val_data.split(0.1)[0]

ani_net = torchani.ANIModel([AtomicNetPaper(), AtomicNetPaper(), AtomicNetPaper(), AtomicNetPaper()])
model = nn.Sequential(aev_computer, ani_net).to(device)

trainer = ANITrainer(model, batch_size=512, learning_rate=1e-2, epoch=60, l2=1e-5)
train_losses, val_losses = trainer.train(exp_train, exp_val)
test_loss = trainer.evaluate(test_data, draw_plot=True)
print(f"Paper Network 60%, 60 epoch, 1e-2 lr test loss: {test_loss:.6f}")
```

```

print(f"Final train loss: {train_losses[-1]:.6f}")
print(f"Final val loss: {val_losses[-1]:.6f}")
print(f"Train/Val gap: {abs(train_losses[-1] - val_losses[-1]):.6f}")

best_epoch = val_losses.index(min(val_losses))
print(f"Best epoch: {best_epoch}/{len(val_losses)}")
print(f"Best val loss: {min(val_losses):.6f}")

improvement = (train_losses[0] - train_losses[-1]) / train_losses[0] * 100
print(f"Train loss improvement: {improvement:.1f}%")

```

Sequential – Number of parameters: 296452

Initialize training data...

100%|██████████| 60/60 [06:34<00:00, 6.57s/it]

Paper Network 60%, 60 epoch, 1e-2 lr test loss: 0.000087

Final train loss: 0.000124

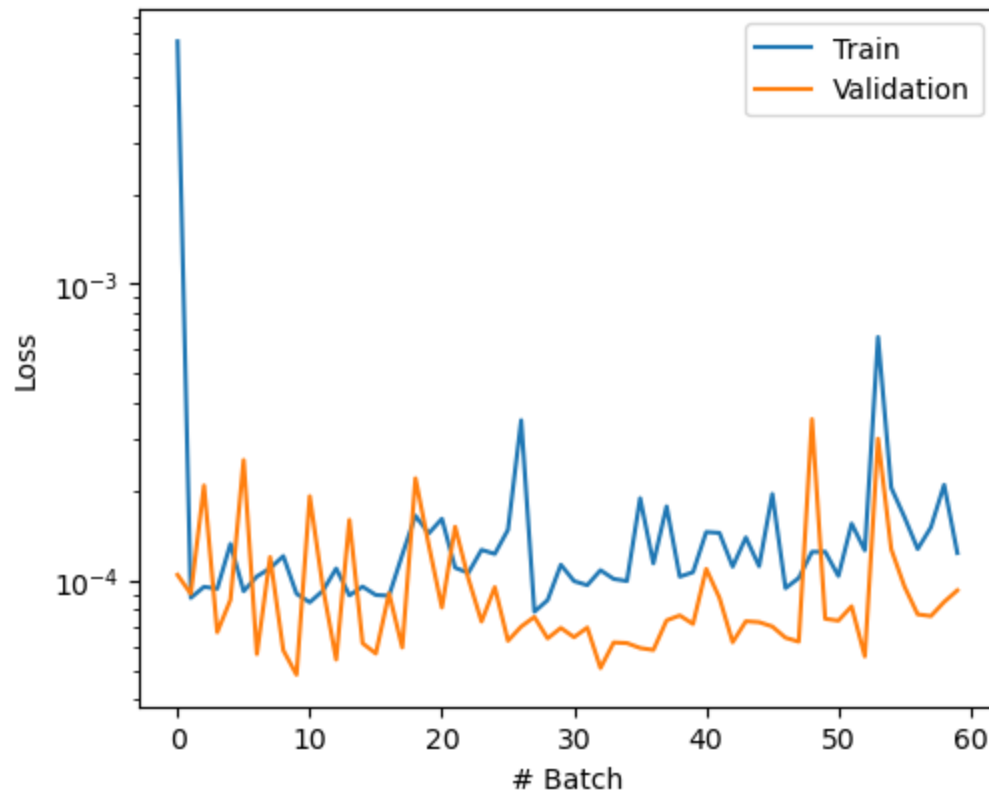
Final val loss: 0.000093

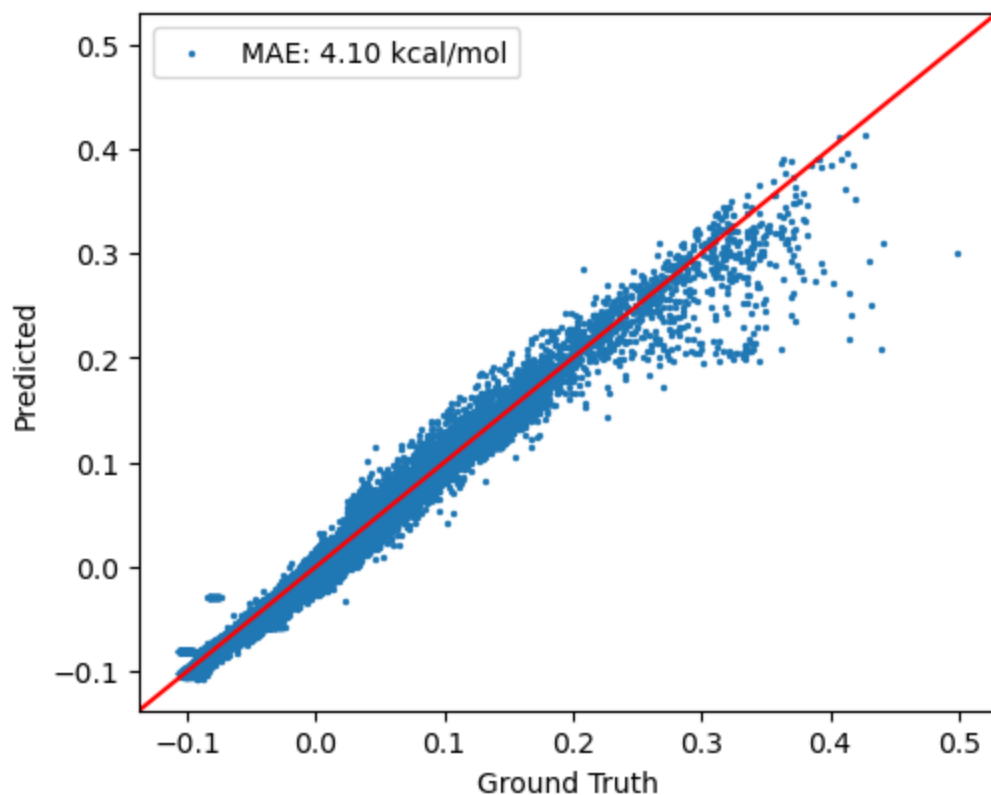
Train/Val gap: 0.000031

Best epoch: 9/60

Best val loss: 0.000048

Train loss improvement: 98.1%





Paper Network 60%, 60 epoch, 1e-2 lr test loss: 0.000087 Final train loss: 0.000124 Final val loss: 0.000093 Train/Val gap: 0.000031 Best epoch: 9/60 Best val loss: 0.000048 Train loss improvement: 98.1%

```
In [19]: #lower learning rate
exp_train = train_data.split(0.6)[0]
exp_val = val_data.split(0.1)[0]

ani_net = torchani.ANIModel([AtomicNetPaper(), AtomicNetPaper(), AtomicNetPaper(), AtomicNetPaper()])
model = nn.Sequential(aev_computer, ani_net).to(device)

trainer = ANITrainer(model, batch_size=512, learning_rate=1e-4, epoch=60, l2=1e-5)
train_losses, val_losses = trainer.train(exp_train, exp_val)
test_loss = trainer.evaluate(test_data, draw_plot=True)
print(f"Paper Network 60%, 60 epoch, 1e-4 lr test loss: {test_loss:.6f}")

print(f"Final train loss: {train_losses[-1]:.6f}")
print(f"Final val loss: {val_losses[-1]:.6f}")
print(f"Train/Val gap: {abs(train_losses[-1] - val_losses[-1]):.6f}")
```



```
best_epoch = val_losses.index(min(val_losses))
print(f"Best epoch: {best_epoch}/{len(val_losses)}")
print(f"Best val loss: {min(val_losses):.6f}")

improvement = (train_losses[0] - train_losses[-1]) / train_losses[0] * 100
print(f"Train loss improvement: {improvement:.1f}%")
```

Sequential - Number of parameters: 296452

Initialize training data...

100%|██████████| 60/60 [06:27<00:00, 6.45s/it]

Paper Network 60%, 60 epoch, 1e-4 lr test loss: 0.000012

Final train loss: 0.000012

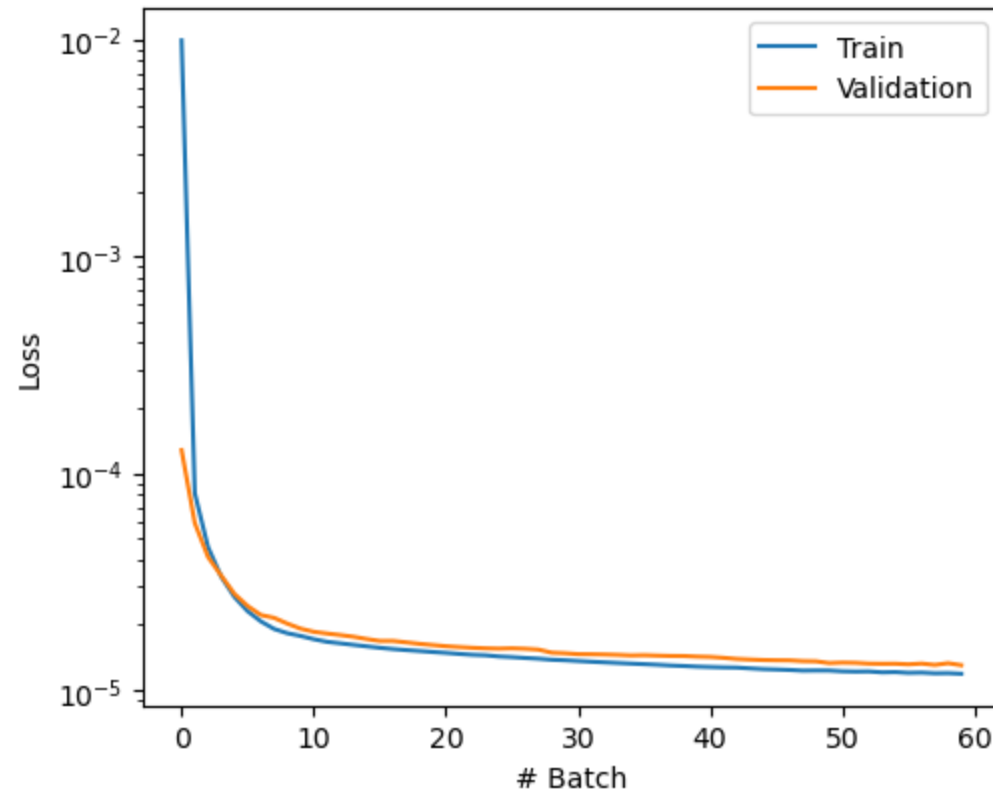
Final val loss: 0.000013

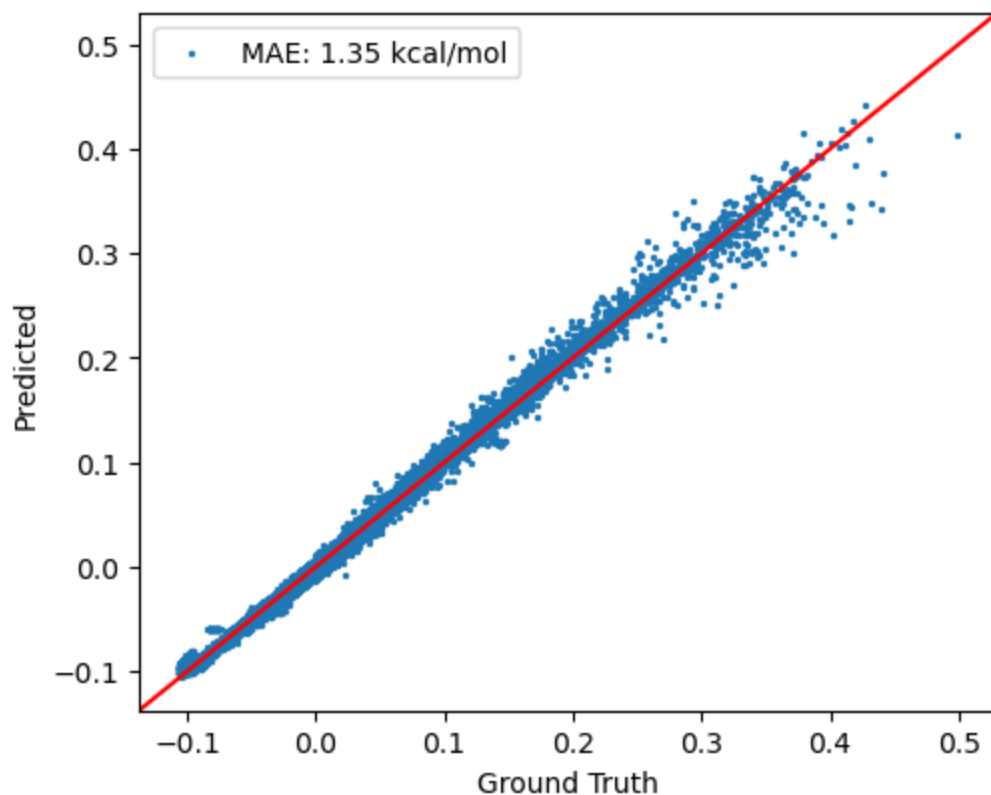
Train/Val gap: 0.000001

Best epoch: 59/60

Best val loss: 0.000013

Train loss improvement: 99.9%





t1: Paper Network 60%, 60 epoch, 1e-4 lr test loss: 0.000013 Final train loss: 0.000012 Final val loss: 0.000014 Train/Val gap: 0.000002 Best epoch: 59/60 Best val loss: 0.000014 Train loss improvement: 99.0% t2: Paper Network 60%, 60 epoch, 1e-4 lr test loss: 0.000013 Final train loss: 0.000013 Final val loss: 0.000014 Train/Val gap: 0.000001 Best epoch: 59/60 Best val loss: 0.000014 Train loss improvement: 100.0% t3: Paper Network 60%, 60 epoch, 1e-4 lr test loss: 0.000012 Final train loss: 0.000012 Final val loss: 0.000013 Train/Val gap: 0.000001 Best epoch: 59/60 Best val loss: 0.000013 Train loss improvement: 99.9%

```
In [22]: # no l2 regularization
exp_train = train_data.split(0.6)[0]
exp_val = val_data.split(0.1)[0]

ani_net = torchani.ANIModel([AtomicNetPaper(), AtomicNetPaper(), AtomicNetPaper(), AtomicNetPaper()])
model = nn.Sequential(aev_computer, ani_net).to(device)

trainer = ANITrainer(model, batch_size=512, learning_rate=1e-4, epoch=60, l2=0)
train_losses, val_losses = trainer.train(exp_train, exp_val)
test_loss = trainer.evaluate(test_data, draw_plot=True)
print(f"Paper Network 60%, 60 epoch, 1e-4 lr test loss: {test_loss:.6f}")
```

```

print(f"Final train loss: {train_losses[-1]:.6f}")
print(f"Final val loss: {val_losses[-1]:.6f}")
print(f"Train/Val gap: {abs(train_losses[-1] - val_losses[-1]):.6f}")

best_epoch = val_losses.index(min(val_losses))
print(f"Best epoch: {best_epoch}/{len(val_losses)}")
print(f"Best val loss: {min(val_losses):.6f}")

improvement = (train_losses[0] - train_losses[-1]) / train_losses[0] * 100
print(f"Train loss improvement: {improvement:.1f}%")

```

Sequential – Number of parameters: 296452

Initialize training data...

100%|██████████| 60/60 [06:25<00:00, 6.43s/it]

Paper Network 60%, 60 epoch, 1e-4 lr test loss: 0.000003

Final train loss: 0.000002

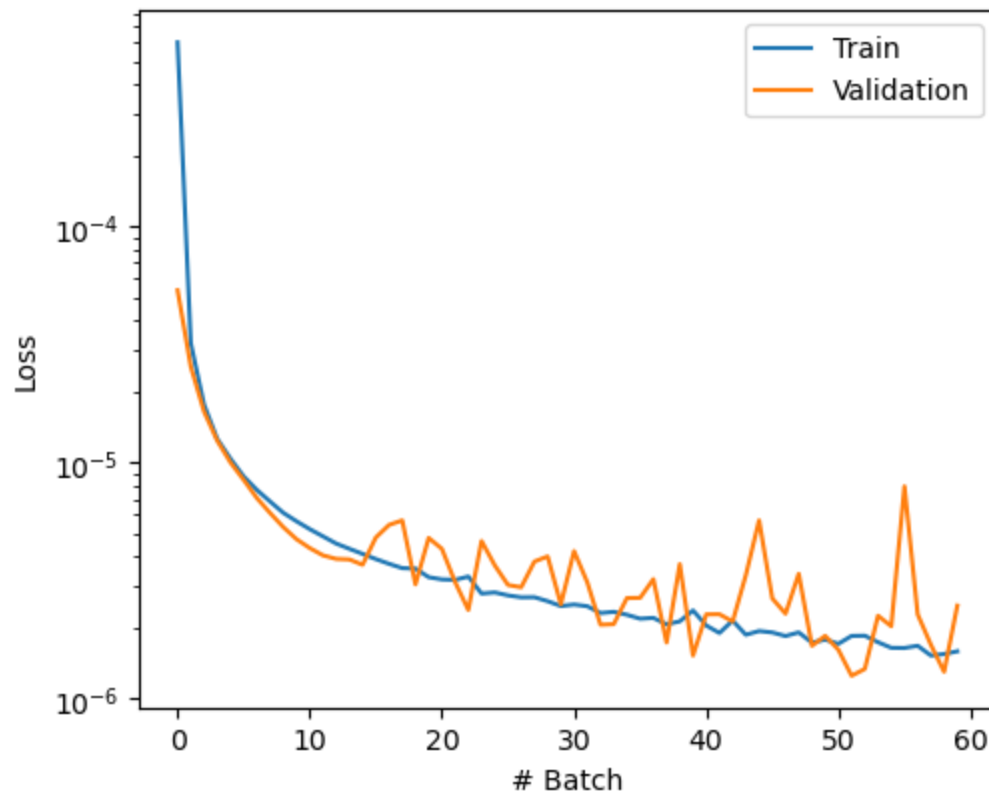
Final val loss: 0.000002

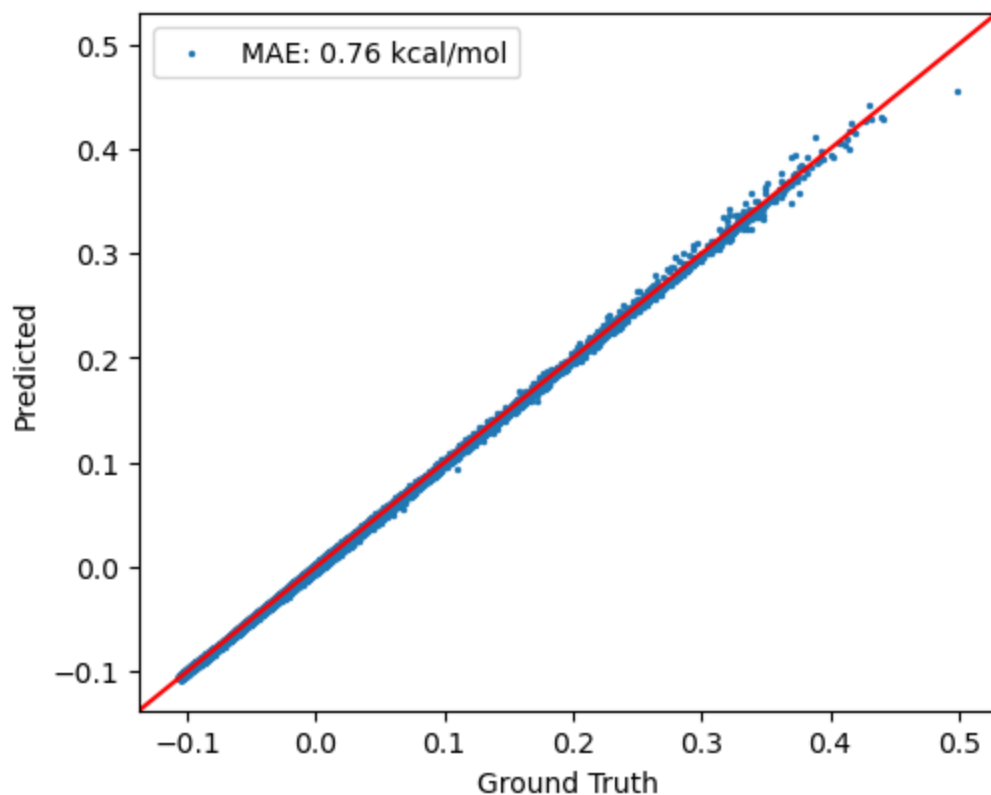
Train/Val gap: 0.000001

Best epoch: 51/60

Best val loss: 0.000001

Train loss improvement: 99.7%





t1: Paper Network 60%, 60 epoch, 1e-4 lr l2=0 test loss: 0.000002 Final train loss: 0.000002 Final val loss: 0.000002 Train/Val gap: 0.000000
 Best epoch: 58/60 Best val loss: 0.000001 Train loss improvement: 99.8% t2: Paper Network 60%, 60 epoch, 1e-4 lr l2=0 test loss:
 0.000002 Final train loss: 0.000002 Final val loss: 0.000002 Train/Val gap: 0.000000 Best epoch: 56/60 Best val loss: 0.000001 Train loss
 improvement: 99.8% t3: Paper Network 60%, 60 epoch, 1e-4 lr test loss: 0.000003 Final train loss: 0.000002 Final val loss: 0.000002
 Train/Val gap: 0.000001 Best epoch: 51/60 Best val loss: 0.000001 Train loss improvement: 99.7%

```
In [ ]: # no l2 regularization, 120 epochs
exp_train = train_data.split(0.6)[0]
exp_val = val_data.split(0.1)[0]

ani_net = torchani.ANIModel([AtomicNetPaper(), AtomicNetPaper(), AtomicNetPaper(), AtomicNetPaper()])
model = nn.Sequential(aev_computer, ani_net).to(device)

trainer = ANITrainer(model, batch_size=512, learning_rate=1e-4, epoch=120, l2=0)
train_losses, val_losses = trainer.train(exp_train, exp_val)
test_loss = trainer.evaluate(test_data, draw_plot=True)
print(f"Paper Network 60%, 60 epoch, 1e-4 lr test loss: {test_loss:.6f}")
```

```
print(f"Final train loss: {train_losses[-1]:.6f}")
print(f"Final val loss: {val_losses[-1]:.6f}")
print(f"Train/Val gap: {abs(train_losses[-1] - val_losses[-1]):.6f}")

best_epoch = val_losses.index(min(val_losses))
print(f"Best epoch: {best_epoch}/{len(val_losses)}")
print(f"Best val loss: {min(val_losses):.6f}")

improvement = (train_losses[0] - train_losses[-1]) / train_losses[0] * 100
print(f"Train loss improvement: {improvement:.1f}%")
```

In savio queue for 8 hours and was unable to run my final tests to determine the appropriate amount of epochs hwoever, based on many of the runs having the last 2-3 epochs being the best, it seems the model has not converged on 60 and 90 epochs so I will try 120 and more if the result remains the same.